Low-Level Design: KitchenFlow Restaurant Order & Kitchen Display System

Document ID: KF-LLD-001 · Version 0.1 (draft) · 30 October 2026 · Author: Rathi · Related: KF-PDD-001, KF-SRS-001, KF-SAD-001, KF-DBD-001, KF-API-001

1. Introduction

1.1 Purpose

This document turns the architecture (SAD), the schema (DBD) and the API contract (API) into class-level design: the package layout, the classes of every module with their responsibilities and key signatures, the algorithms of the risky operations, how errors, security, rate limiting, caching, events and streams are built, and an ordered build plan. It is document 6 of 7, and it is written so that you can code from it. The code blocks are Java 21 skeletons: they show structure and the critical lines, not every import, getter or trivial CRUD method.

1.2 Scope

The backend only. The React screens follow the API and are outside the seven documents. Technology: Spring Boot 3, Spring Security (resource server), Spring Data JPA with Hibernate, JdbcTemplate for native statements and reports, Flyway, Caffeine, Bucket4j and Micrometer.

1.3 Conventions

Topic	Rule
Package root	com.kitchenflow, one package per module, each split into api, application, domain and infrastructure (SAD 4.3)
Names	*Controller in api; *Service and *Workflow in application; *Repository in infrastructure; *Request and *View are records; *Event are records
DTOs and entities	DTOs are records in api and never contain entities. Entities stay in domain and infrastructure and never leave the application layer

Topic	Rule
Transactions	Only application services open them (@Transactional). Controllers never do
Time	Always from an injected Clock; no call to Instant.now() in business code
Statements	S1 to S14 refer to DBD section 8 and its proposed additions
Comments in code	A comment such as SAD 6.2 points to the design rule behind the line
Library versions	Snippets target Spring Boot 3, Spring Security 6, Bucket4j 8 and Caffeine 3. Check each against the version you pin (SAD open item AI-01)

2. Project structure

2.1 Repository layout

```
kitchenflow/
  pom.xml
  Dockerfile, docker-compose.yml, .env.example
  nginx/nginx.conf
  ui/                                  React app (built into the Nginx
image)
  db/ops/roles.sql                     run by Compose init (DBD section 4)
  src/main/resources/
    application.yml, application-dev.yml, application-compose.yml,
    application-loadtest.yml
    db/migration/V1__baseline.sql      Flyway location (DBD section 5)
    db/seed/seed_demo.sql, gen_orders_100k.sql
  src/main/java/com/kitchenflow/       see 2.2
  src/test/java/com/kitchenflow/       mirrors main, plus architecture and
concurrency tests
  .github/workflows/ci.yml
```

2.2 Packages and classes

```
com.kitchenflow
  KitchenFlowApplication
  common/        Money, Actor, Page envelope (PageView), CursorCodec
  platform/
    config/      JacksonConfig, SecurityConfig, WebConfig, AsyncConfig,
KitchenFlowProperties
    error/       ErrorCode, ApiException, FieldError, ApiError,
GlobalExceptionHandler, ConstraintMap
    web/         CorrelationFilter, ContentLengthFilter, AllowedParams,
UnknownParamInterceptor,
                 TrimmingStringDeserializer, NoControlChars, MoneyJson
    ratelimit/   RateClass, RateLimitPolicy, RateLimitFilter,
LoginRateLimiter
    security/    AuthenticatedUser, JwtService, JwtAuthConverter,
StationAccess
    sse/         SseHub, SseSubscriber, StreamKey, SseEvent
    time/        ClockConfig, BusinessDayService
    tx/          DeadlockRetry
    jobs/        UnpaidOrderJob, PurgeJobs, LateTicketScanner
  auth/          api: AuthController, UserAdminController; application:
AuthService, LockoutService,
                 RefreshTokenService, UserAdminService; domain: AppUser,
Role; infrastructure: repositories
  config/        ConfigService (cached settings), StationService,
ConfigController, StationController
  menu/          MenuController, AdminCatalogController; MenuService,
MenuQuery, MenuCache, ComboService;
                 domain: Category, MenuItem, ModifierGroup, Modifier, Combo
  inventory/     StockController; StockService; domain: ItemStock,
StockAdjustment
  order/         OrderController; OrderService, OrderEditService,
OrderCancelService, OrderWorkflow,
                 OrderSearchService, PricingService, TokenService,
IdempotencyService, OrderValidator;
                 domain: Order, OrderLine, OrderStatus, OrderStateMachine;
spi: TicketGateway
  payment/       PaymentController; PaymentService, BillService,
RefundService; domain: Payment, Refund
  kitchen/       TicketController, KitchenStreamController; TicketService,
TicketFactory,
                 KitchenTicketGateway, SlaCalculator, KitchenOverviewService;
```

```
domain: Ticket, TicketStatus
  board/         BoardController; BoardService (1-second snapshot cache)
  report/        ReportController; ReportService, ReportBulkhead
  audit/         AuditController; AuditService, AuditSearchService; domain:
AuditEvent, AuditAction
```

Dependency directions follow SAD Figure 2: payment depends on order; kitchen depends on order and implements order.spi.TicketGateway; order depends on menu, inventory and audit; inventory depends on menu.

2.3 Configuration properties

One typed, validated properties record carries every limit that SAD 13.2 lists. Nothing reads a limit from a literal in code.

```
@ConfigurationProperties(prefix = "kitchenflow")
@Validated
public record KitchenFlowProperties(
    @NotNull Jwt jwt, @NotNull RateLimits rateLimit, @NotNull Cache cache,
@NotEmpty List<String> corsOrigins) {

  public record Jwt(@NotBlank String secret, @NotNull Duration accessTtl,
@NotNull Duration refreshTtl) {}
  public record Bucket(@Min(1) int capacity, @Min(1) int refill, @NotNull
Duration period) {}
  public record RateLimits(Bucket login, Bucket refresh, Bucket orderWrite,
Bucket ticketAction, Bucket read,
                           Bucket report, Bucket adminWrite, Bucket
boardRead, Bucket sseConnect, Sse sse) {}
  public record Sse(int perUser, int perAddress, int total) {}
  public record Cache(@NotNull Duration menuView, @NotNull Duration
appConfig) {}
}
```

The JWT secret must decode to at least 256 bits; start-up fails otherwise. The loadtest profile loads larger buckets, and a start-up check refuses that profile when the compose profile is also active (SAD 5.2).

3. Platform building blocks

These classes serve every module. Build them first (section 7, steps 1 to 6).

3.1 Money

Money is a value object over BigDecimal with scale 2 (DD-04, BR-02). Rounding is HALF_UP, which matches the database's round(numeric, 2) for the non-negative amounts used here, so Java and the CHECK constraints in DBD 5.7 always agree.

```
public record Money(BigDecimal amount) implements Comparable<Money> {
    private static final BigDecimal HUNDRED = BigDecimal.valueOf(100);
    public static final Money ZERO = new Money(new BigDecimal("0.00"));

    public Money {                                          // compact
constructor
        Objects.requireNonNull(amount);
        if (amount.scale() != 2) throw new IllegalArgumentException("Money
needs scale 2: " + amount);
    }
    public static Money parse(String text) { return new Money(new
BigDecimal(text)); }
    public static Money round(BigDecimal v) { return new Money(v.setScale(2,
RoundingMode.HALF_UP)); }

    public Money plus(Money o)   { return new Money(amount.add(o.amount)); }
    public Money minus(Money o)  { return new
Money(amount.subtract(o.amount)); }
    public Money times(int qty)  { return new
Money(amount.multiply(BigDecimal.valueOf(qty))); }
    public Money percent(BigDecimal rate) {                 // BR-03: GST
on one line
        return round(amount.multiply(rate).divide(HUNDRED));  // dividing by
100 always terminates
    }
    @Override public int compareTo(Money o) { return
amount.compareTo(o.amount); }
    @Override public String toString() { return amount.toPlainString(); }
}
```

On the wire Money is a JSON string (API 2.1). MoneyJson has a serializer that writes toString() and a deserializer that accepts only a string token: a number is rejected as a type mismatch (400), a string with other than two decimals fails with INVALID_SCALE (422), and any other text with INVALID_FORMAT. A signed variant of the deserializer, selected with @JsonDeserialize on priceDelta fields, allows a leading minus.

3.2 Clock and business day

```
@Configuration class ClockConfig { @Bean Clock clock() { return
Clock.systemUTC(); } }   // tests inject a fixed Clock

@Component
public class BusinessDayService {                                  // BR-16
    private static final ZoneId IST = ZoneId.of("Asia/Kolkata");
    private final Clock clock;
    private final ConfigService config;

    public LocalDate businessDayOf(Instant at) {
        ZonedDateTime local = at.atZone(IST);
        return local.toLocalTime().isBefore(config.businessDayCutoff())
            ? local.toLocalDate().minusDays(1)
            : local.toLocalDate();
    }
    public LocalDate currentBusinessDay() { return
businessDayOf(clock.instant()); }
}
```

Boundary test: with the cutoff at 04:00, 03:59:59 on 31 October belongs to business day 30 October, and 04:00:00 belongs to 31 October.

3.3 Error model

Every failure becomes an ApiException or is translated into one, and one handler writes the envelope of API 2.4.

```
public enum ErrorCode {
    MALFORMED_REQUEST(400), UNAUTHENTICATED(401), FORBIDDEN(403),
NOT_FOUND(404),
    METHOD_NOT_ALLOWED(405), NOT_ACCEPTABLE(406), INVALID_TRANSITION(409),
STALE_VERSION(409),
    OUT_OF_STOCK(409), PAYMENT_ALREADY_RECORDED(409), RESOURCE_IN_USE(409),
    PAYLOAD_TOO_LARGE(413), UNSUPPORTED_MEDIA_TYPE(415),
VALIDATION_FAILED(422),
    ITEM_UNAVAILABLE(422), AMOUNT_MISMATCH(422), IDEMPOTENCY_KEY_REUSED(422),
    RATE_LIMITED(429), INTERNAL_ERROR(500), SERVICE_BUSY(503);
    private final int status;
    ErrorCode(int status) { this.status = status; }
    public int status() { return status; }
}
```

```
public record FieldError(String field, String code, String message) {}
public record ApiError(Instant timestamp, int status, String code, String
message, String path,
                       String correlationId, List<FieldError> errors,
Map<String, Object> details) {}

public class ApiException extends RuntimeException {
    private final ErrorCode code;
    private final List<FieldError> errors;        // only VALIDATION_FAILED
and ITEM_UNAVAILABLE
    private final Map<String, Object> details;    // API 2.4: current, items,
expectedAmount, ...
    private final Map<String, String> headers;    // for example Retry-After

    public static ApiException validation(List<FieldError> errors) {       //
API 9.1 rule
        boolean onlyAvailability = errors.stream().allMatch(e ->
e.code().equals("ITEM_UNAVAILABLE"));
        return new ApiException(onlyAvailability ? ErrorCode.ITEM_UNAVAILABLE
: ErrorCode.VALIDATION_FAILED,
                                "The request has " + errors.size() + "
invalid fields.", errors, Map.of(), Map.of());
    }
    public static ApiException invalidTransition(String currentStatus, String
operation, String reason) { ... }
    public static ApiException staleVersion(Object current) { ... }
    public static ApiException outOfStock(List<StockShortage> items) { ... }
    public static ApiException rateLimited(String rateClass, long
retryAfterSeconds) { ... }
}
```

GlobalExceptionHandler (a @RestControllerAdvice) translates the framework's exceptions:

Exception	Code	Notes
ApiException	Its own code	Headers such as Retry-After are copied
MethodArgumentNotValidException, HandlerMethodValidationException, ConstraintViolationException	VALIDATION_FAILED	Each violation becomes a FieldError (table below)

Exception	Code	Notes
HttpMessageNotReadableException	MALFORMED_REQUEST	An unknown property names the property. An invalid enum value becomes VALIDATION_FAILED with INVALID_ENUM; a money scale error becomes INVALID_SCALE
MethodArgumentTypeMismatchException, malformed parameter	MALFORMED_REQUEST	Non-numeric path id
HttpRequestMethodNotSupportedException	METHOD_NOT_ALLOWED	Allow header set
HttpMediaTypeNotSupportedException	UNSUPPORTED_MEDIA_TYPE	
HttpMediaTypeNotAcceptableException	NOT_ACCEPTABLE	
NoResourceFoundException	NOT_FOUND	After authentication
AccessDeniedException	FORBIDDEN	From method security
ObjectOptimisticLockingFailureException	STALE_VERSION	Loads the current representation in a new read-only transaction for details.current
PessimisticLockingFailureException	SERVICE_BUSY, reason LOCK_TIMEOUT	Raised after DeadlockRetry has given up
QueryTimeoutException	SERVICE_BUSY, reason QUERY_TIMEOUT	SQLState 57014
CannotCreateTransactionException, SQLTransientConnectionException	SERVICE_BUSY, reason POOL	Retry-After 1
DataIntegrityViolationException	See ConstraintMap	
Throwable	INTERNAL_ERROR	Generic message; the stack trace and correlation id go to the log only

Bean Validation annotation	Field code
NotNull, NotBlank	REQUIRED

Bean Validation annotation	Field code
Size on a string	TOO_SHORT or TOO_LONG
Size on a collection	TOO_FEW or TOO_MANY
Min, Max, DecimalMin, DecimalMax, Range	OUT_OF_RANGE
Pattern	INVALID_FORMAT
NoControlChars (custom)	INVALID_CHARACTERS

Database errors are translated through ConstraintMap, keyed by the constraint names of the DBD, so a race that slips past validation still gives the documented answer:

```
@ExceptionHandler(DataIntegrityViolationException.class)
ResponseEntity<ApiError> onIntegrity(DataIntegrityViolationException ex,
HttpServletRequest req) {
    String constraint = ConstraintMap.constraintOf(ex);      // Hibernate
ConstraintViolationException#getConstraintName
    return switch (ConstraintMap.sqlStateOf(ex)) {
        case "23505" -> respond(ConstraintMap.forUnique(constraint), req);
// duplicate or already recorded
        case "23503" -> respond(ConstraintMap.forForeignKey(constraint),
req); // delete blocked: RESOURCE_IN_USE
        default      -> respondInternal(ex, req);
// 23514 check violation = a defect
    };
}
```

Constraint	Result
uq_payment_order	409 PAYMENT_ALREADY_RECORDED
uq_app_user_username, uq_station_name, uq_category_name, uq_menu_item_name, uq_combo_name, uq_modifier_name	422 VALIDATION_FAILED, DUPLICATE_VALUE on name or username
fk_ol_item, fk_olc_item, fk_ol_combo, fk_tki_item, fk_ci_item, fk_adj_item, fk_olm_modifier on a delete	409 RESOURCE_IN_USE with references by type (orders, tickets, combos, stock history)

Constraint	Result
uq_orders_day_token, uq_orders_number, uq_ticket_order_station, uq_ol_order_line, uq_idem_user_key, uq_rt_hash	500 INTERNAL_ERROR: these cannot happen unless code is wrong, so they are logged at ERROR
Any ck_ constraint	500 INTERNAL_ERROR, logged at ERROR: validation should have stopped it

Filters and security handlers run outside MVC, so they cannot use the advice. They call ErrorResponseWriter.write(response, apiException), which produces the identical envelope.

3.4 JSON and input strictness

Strictness comes from configuration plus a small Jackson module (API 2.1):

```
spring:
  jackson:
    deserialization:
      fail-on-unknown-properties: true      # unknown JSON property -> 400
      fail-on-null-for-primitives: true
      accept-float-as-int: false            # 2.5 is not a valid quantity
    mapper:
      allow-coercion-of-scalars: false      # the string "2" is not the
number 2
    serialization:
      write-dates-as-timestamps: false
    default-property-inclusion: always      # nulls are written, not omitted
  mvc:
    problemdetails:
      enabled: false                        # we write our own envelope
```

```
@Bean Module kitchenFlowModule() {
    SimpleModule m = new SimpleModule("kitchenflow");
    m.addSerializer(Instant.class, new InstantMillisSerializer());   // 2026-
10-30T09:15:30.123Z, always 3 digits
    m.addSerializer(Money.class, new MoneyJson.Serializer());
    m.addDeserializer(Money.class, new MoneyJson.Deserializer());
    m.addDeserializer(String.class, new TrimmingStringDeserializer());
    return m;
}
```

InstantMillisSerializer formats with the pattern yyyy-MM-dd'T'HH:mm:ss.SSS'Z' in UTC, because the default Instant output drops trailing zeros. TrimmingStringDeserializer accepts only string tokens, trims, and turns a blank into null so Bean Validation reports REQUIRED. The custom constraint NoControlChars rejects control characters.

3.5 Filters and interceptors

```
@Component @Order(Ordered.HIGHEST_PRECEDENCE)
public class CorrelationFilter extends OncePerRequestFilter {
    private static final Pattern VALID = Pattern.compile("[A-Za-z0-9._-]
{1,64}");
    @Override protected void doFilterInternal(HttpServletRequest req,
HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String id = req.getHeader("X-Correlation-Id");
        if (id == null || !VALID.matcher(id).matches()) id =
UUID.randomUUID().toString();
        MDC.put("correlationId", id);
        res.setHeader("X-Correlation-Id", id);
        try { chain.doFilter(req, res); } finally { MDC.clear(); }
    }
}

@Target(METHOD) @Retention(RUNTIME) public @interface AllowedParams {
String[] value(); }

public class UnknownParamInterceptor implements HandlerInterceptor {      //
API 2.1: unknown parameter -> 400
    @Override public boolean preHandle(HttpServletRequest req,
HttpServletResponse res, Object handler) {
        if (handler instanceof HandlerMethod hm) {
            AllowedParams allowed =
hm.getMethodAnnotation(AllowedParams.class);
            Set<String> names = allowed == null ? Set.of() :
Set.of(allowed.value());
            for (String p : req.getParameterMap().keySet())
                if (!names.contains(p)) throw ApiException.malformed("Unknown
query parameter: " + p);
        }
        return true;
    }
}
```

ContentLengthFilter answers 413 PAYLOAD_TOO_LARGE when Content-Length exceeds 1 MB, and for chunked bodies wraps the input stream in a counter that fails at the limit. Nginx enforces the same limit first (client_max_body_size 1m).

3.6 Security

Components

Class	Role
AuthenticatedUser	record (id, role, stationIds): the principal placed in the security context
JwtService	Issues the 15-minute access token
JwtAuthConverter	Converts a validated Jwt into a token whose principal is an AuthenticatedUser and whose authority is ROLE_ plus the role
StationAccess	Station binding check for Kitchen users
PasswordPolicy	Length in bytes (8 to 72), not equal to the username, not on the common-password list
LockoutService, RefreshTokenService	In section 4.1

```
public String issue(AppUser u, Collection<Long> stationIds) {
    Instant now = clock.instant();
    JwtClaimsSet claims = JwtClaimsSet.builder()
        .subject(String.valueOf(u.getId()))
        .issuedAt(now).expiresAt(now.plus(props.jwt().accessTtl()))
        .id(UUID.randomUUID().toString())
        .claim("role", u.getRole().name())
        .claim("stations", stationIds)
        .build();
    return
encoder.encode(JwtEncoderParameters.from(JwsHeader.with(MacAlgorithm.HS256).b
uild(), claims))
                   .getTokenValue();
}
```

The decoder is NimbusJwtDecoder.withSecretKey(key).macAlgorithm(MacAlgorithm.HS256), which checks the signature and expiry. The key is the Base64-decoded secret from the environment.

URL-level authorization is the primary role gate. Method security (@PreAuthorize) would run only after Spring has parsed the body and validated it, so a forbidden caller could learn validation messages. The role rules therefore live in the filter chain, where a 403 happens before any parsing (API 2.5, step 5). @PreAuthorize stays as a second layer (SAD 10.3).

```
http.csrf(AbstractHttpConfigurer::disable)        // bearer tokens; the
cookie endpoints check Origin instead
    .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .cors(Customizer.withDefaults())
    .authorizeHttpRequests(a -> a
        .requestMatchers(POST, "/api/v1/auth/login", "/api/v1/auth/refresh",
"/api/v1/auth/logout").permitAll()
        .requestMatchers(GET, "/api/v1/board", "/api/v1/stream/board",
"/actuator/health/**").permitAll()
        .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
        .requestMatchers(POST, "/api/v1/orders", "/api/v1/orders/*/payments",
                         "/api/v1/orders/*/cancel",
"/api/v1/orders/*/complete").hasAnyRole("CASHIER", "MANAGER")
        .requestMatchers(PATCH, "/api/v1/orders/*/rush").hasRole("MANAGER")
        // ... the remaining rows of the table below ...
        .anyRequest().authenticated())
    .oauth2ResourceServer(o -> o.jwt(j ->
j.jwtAuthenticationConverter(jwtAuthConverter))
                                .authenticationEntryPoint(entryPoint))
// 401 envelope, WWW-Authenticate: Bearer
    .exceptionHandling(e -> e.accessDeniedHandler(deniedHandler))
// 403 envelope
    .addFilterAfter(rateLimitFilter, BearerTokenAuthenticationFilter.class);
```

Pattern	Allowed roles
POST /auth/login, /auth/refresh, /auth/logout; GET /board, /stream/board, health	Public

Pattern	Allowed roles
GET /auth/me; POST /auth/change-password; GET /stations; GET /menu	Any authenticated user
/admin/**	ADMIN
PATCH /items/*/availability; /stock; /items/*/stock-adjustments	MANAGER, ADMIN
POST and PUT /orders and sub-resources; GET /orders and sub-resources	CASHIER, MANAGER
PATCH /orders/*/rush; GET /refunds; POST /refunds/*/complete; GET /kitchen/overview; GET /stream/manager	MANAGER
GET /stations/*/tickets; /tickets/**; GET /stream/stations/*	KITCHEN, MANAGER
GET /reports/**; GET /audit	MANAGER, ADMIN
GET /actuator/prometheus	ADMIN

The last rule is anyRequest().authenticated(), so unknown paths reach MVC and return 404 as the API requires. The price of that choice is that a new endpoint without a rule would be open to any authenticated user. A build-time test, EndpointRuleCoverageTest, reads every controller mapping and fails if one has no explicit rule in the table.

Rules that depend on data run inside services, after the lookup: a Cashier cancelling an IN_KITCHEN order gets 403 (API 9.5), and a Kitchen user at an unbound station gets 403 on a ticket (the station comes from the ticket). For paths that carry the station id (K-01, X-01), StationAccess runs before any lookup:

```
public void require(AuthenticatedUser user, long stationId) {
    if (user.role() == Role.MANAGER) return;
    if (!user.stationIds().contains(stationId)) throw
ApiException.forbidden("Station not assigned to you");
}
```

Ordering note. In Spring the authentication filter runs before the MVC syntax checks (405, 415 and body parsing). For protected endpoints the real order is therefore 401, then the URL-level 403, then the rate limit, then 405, 415 and 400, and then validation. Section 8 proposes updating API section 2.5 to match.

3.7 Rate limiting

One filter, placed after authentication (so the user is known), applies the application layer of SAD 5.2. Buckets come from KitchenFlowProperties; nothing is hard-coded.

RateClass	Matches (method and path)	Key
REFRESH	POST /auth/refresh, /auth/logout, /auth/change-password	User or address
ORDER_WRITE	POST, PUT, PATCH on /orders/**; POST /orders/*/payments; POST /refunds*/*/complete	User
TICKET_ACTION	POST /tickets/*/start, bump, recall, acknowledge-cancel	User
READ	Other GET requests, except streams, reports, audit and the board	User
REPORT	GET /reports/**, GET /audit	User
ADMIN_WRITE	POST, PUT, PATCH, DELETE on /admin/** and /items/**	User
BOARD_READ	GET /board	Client address
SSE_CONNECT	GET /stream/**	User, or address for the board
LOGIN	POST /auth/login	Address and username; applied inside AuthController (the username is in the body), see 4.1

The client address is request.getRemoteAddr(). Tomcat's RemoteIpValve replaces it with the real client address only when the request comes from the Nginx container, which is the only trusted proxy (server.forward-headers-strategy set to native and server.tomcat.remoteip.internal-proxies set to the Compose network). Without that restriction a client could forge X-Forwarded-For and escape its limit.

```
@Override protected void doFilterInternal(HttpServletRequest req,
HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
    Optional<RateClass> cls = policy.classify(req);                 //
```

```
empty: health, static files
    if (cls.isEmpty()) { chain.doFilter(req, res); return; }
    try {
        String key = cls.get() + "|" + policy.keyOf(cls.get(), req);   //
user id or client address
        Bucket bucket = buckets.get(key, k ->
newBucket(props.rateLimit().of(cls.get())));
        ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);
        res.setHeader("X-RateLimit-Limit",
String.valueOf(props.rateLimit().of(cls.get()).capacity()));
        res.setHeader("X-RateLimit-Remaining",
String.valueOf(probe.getRemainingTokens()));
        if (!probe.isConsumed()) {
            long wait = Math.max(1,
TimeUnit.NANOSECONDS.toSeconds(probe.getNanosToWaitForRefill()) + 1);
            metrics.counter("kf_rate_limit_rejected_total", "class",
cls.get().name()).increment();
            ErrorResponseWriter.write(res,
ApiException.rateLimited(cls.get().name(), wait));
            return;
        }
    } catch (RuntimeException e) {                                    // fail
open for everything except LOGIN (SAD 5.2)
        log.error("rate limiter failed, request allowed", e);
        metrics.counter("kf_rate_limiter_errors_total").increment();
    }
    chain.doFilter(req, res);
}

private Bucket newBucket(KitchenFlowProperties.Bucket c) {          //
Bucket4j 8.x builder; check your pinned version
    return Bucket.builder().addLimit(l ->
l.capacity(c.capacity()).refillGreedy(c.refill(), c.period())).build();
}
```

The bucket store is a Caffeine cache with expireAfterAccess of 10 minutes and a maximum of 10,000 entries, so a flood of distinct keys cannot exhaust memory. A chain.doFilter call is kept outside the try block so an exception from the controller is never mistaken for a limiter failure.

3.8 Caching

```
public record MenuKey(Long categoryId, String q, boolean includeUnavailable)
{}   // q normalised to lower case

@Component
public class MenuCache {
    private final LoadingCache<MenuKey, MenuView> cache;
    MenuCache(MenuQuery reader, KitchenFlowProperties props, MeterRegistry
meters) {
        cache = Caffeine.newBuilder()
            .expireAfterWrite(props.cache().menuView())      // 5 minutes:
the backstop
            .maximumSize(64)                                  // few filter
combinations exist
            .recordStats()
            .build(reader::load);                             // loads once
per key, even under a stampede
        CaffeineMetrics.monitor(meters, cache, "menuView");
    }
    public MenuView get(MenuKey key) { return cache.get(key); }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    void onMenuChanged(MenuChanged e) { cache.invalidateAll(); }   // SAD
5.3: evict after commit
}
```

The menu controller builds the ETag from the view's menuVersion and the key, and lets Spring answer 304:

```
@GetMapping("/menu")
@AllowedParams({"categoryId", "q", "includeUnavailable"})
ResponseEntity<MenuView> menu(MenuFilter f, ServletWebRequest request) {
    MenuView view = cache.get(f.toKey());
    String etag = view.menuVersion() + "-" +
Integer.toHexString(f.toKey().hashCode());
    if (request.checkNotModified(etag)) return null;                // 304
with no body
    return
ResponseEntity.ok().eTag(etag).cacheControl(CacheControl.noCache()).body(view
);
}
```

Two more caches follow the same pattern. ConfigService keeps the settings in a one-entry LoadingCache (expireAfterWrite of 60 seconds) evicted after commit on ConfigChanged; BoardService keeps the board snapshot for 1 second, because the board is public and a crowd of pollers must not turn into a crowd of queries. Orders, tickets, payments and stock are never cached (SAD 5.3).

3.9 Deadlock retry

A deadlock is retried by running the whole transaction again. The retry must therefore sit outside the transaction: controllers call it, and the service method it wraps opens a fresh transaction on every attempt. Never call DeadlockRetry from inside a @Transactional method.

```
@Component
public class DeadlockRetry {
    private static final int MAX_RETRIES = 2;
    public <T> T run(Supplier<T> transactionalCall) {
        for (int attempt = 0; ; attempt++) {
            try { return transactionalCall.get(); }
            catch (PessimisticLockingFailureException e) {
                if (!isDeadlock(e) || attempt == MAX_RETRIES) throw e;
// lock timeout (55P03) is not retried
                retries.increment();
// kf_db_lock_retry_total
                sleepMillis(ThreadLocalRandom.current().nextInt(20, 61));
// jitter
            }
        }
    }
    private boolean isDeadlock(Throwable e) { /* SQLState 40P01 anywhere in
the cause chain */ }
}
```

Usage: return retry.run(() -> orderService.create(request, key, user));

3.10 Audit

```
@Service
public class AuditService {
    @Transactional(propagation = Propagation.MANDATORY)        // FR-AUD-04:
refuses to run outside a business transaction
    public void record(Actor actor, AuditAction action, EntityType type, long
entityId,
                       String from, String to, String reason, Map<String,
Object> details) {
        repo.save(new AuditEvent(clock.instant(), actor.id(), actor.role(),
type, entityId,
                                 action.name(), from, to, reason,
MDC.get("correlationId"), details));
    }
}
```

MANDATORY propagation is the guard that makes it impossible to write an audit record without a surrounding business transaction. Actor is a record of id and role with a static system() factory (role SYSTEM, id null). AuditEvent maps details to JSONB with @JdbcTypeCode(SqlTypes.JSON), uses the seq_audit_event sequence with allocation size 50, and has no setters (DD-02).

3.11 Domain events and after-commit delivery

```
public record TicketCreated(TicketView ticket) {}
public record TicketChanged(TicketView ticket) {}
public record TicketCancelled(TicketView ticket) {}
public record OrderStatusChanged(OrderSummary order) {}
public record MenuChanged() {}
public record ConfigChanged() {}
public record StockDepleted(long itemId) {}                 // handled inside
the transaction

@Component
class StationStreamListener {
    @Async("eventExecutor")
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    void on(TicketChanged e) {
        try {
            hub.publish(StreamKey.station(e.ticket().stationId()),
"TICKET_CHANGED", e.ticket());
            hub.publish(StreamKey.MANAGER, "TICKET_CHANGED", e.ticket());
        } catch (RuntimeException ex) { log.warn("stream publish failed",
ex); }   // never throws into the business flow
    }
}
```

Rules: events carry immutable views, never entities (an entity may be detached or lazy after commit); a view is built before publishing and includes the version. The eventExecutor is bounded (core 2, maximum 4, queue 1,000). When it rejects a task, its rejection handler calls hub.requestResyncAll(OVERLOAD), which sends SNAPSHOT_REQUIRED to every subscriber, so an overloaded server never loses an event silently. Ordering across threads is not guaranteed, and the version merge on the client makes that harmless (API 12.2).

3.12 The SSE hub

```
public record StreamKey(String type, long id) {
    public static StreamKey station(long id) { return new
StreamKey("STATION", id); }
    public static final StreamKey MANAGER = new StreamKey("MANAGER", 0);
    public static final StreamKey BOARD   = new StreamKey("BOARD", 0);
}
```

```
@Component
public class SseHub {
    private final Map<StreamKey, Set<SseSubscriber>> subscribers = new
ConcurrentHashMap<>();
    private final Map<String, AtomicInteger> perUser = new
ConcurrentHashMap<>(), perAddress = new ConcurrentHashMap<>();
    private final AtomicInteger total = new AtomicInteger();
    private final AtomicLong sequence = new AtomicLong();
    private final ExecutorService sender;             // 2 to 4 threads
    private final ScheduledExecutorService timers;    // heartbeat every 15
s; token-expiry closing

    public SseEmitter subscribe(StreamKey key, String user, String address,
Instant expiresAt) {
        admit(user, address);                          // caps 3 / 20 / 50:
RATE_LIMITED, rateClass CONNECTION_CAP
        SseEmitter emitter = new SseEmitter(Duration.between(clock.instant(),
expiresAt).toMillis() + 5_000);
        SseSubscriber s = new SseSubscriber(emitter, 100, sender);
        subscribers.computeIfAbsent(key, k ->
ConcurrentHashMap.newKeySet()).add(s);
        emitter.onCompletion(() -> release(key, s, user, address));
        emitter.onTimeout(emitter::complete);
        emitter.onError(e -> release(key, s, user, address));
        s.enqueue(next("hello", Map.of("serverTime", clock.instant(),
"stream", key.type(),
                                       "heartbeatSeconds", 15, "expiresAt",
expiresAt)));
        timers.schedule(() -> s.closeWith(next("STREAM_CLOSING",
Map.of("reason", "TOKEN_EXPIRED"))),
                        Duration.between(clock.instant(),
expiresAt).toMillis(), TimeUnit.MILLISECONDS);
        return emitter;
    }

    public void publish(StreamKey key, String name, Object payload) {
        SseEvent e = next(name, payload);
        for (SseSubscriber s : subscribers.getOrDefault(key, Set.of()))
s.enqueue(e);   // overflow closes that one only
    }
    public void requestResyncAll(String reason) { /* SNAPSHOT_REQUIRED to
all, then close */ }
}
```

```
final class SseSubscriber {
    private final SseEmitter emitter;
    private final BlockingQueue<SseEvent> queue;         // bounded at 100
    private final AtomicBoolean draining = new AtomicBoolean();

    boolean enqueue(SseEvent e) {
        if (!queue.offer(e)) { close(); return false; }  // overflow: close,
never skip (SAD 9.4)
        scheduleDrain();
        return true;
    }
    private void scheduleDrain() { if (draining.compareAndSet(false, true))
executor.execute(this::drain); }
    private void drain() {
        try {
            SseEvent e;
            while ((e = queue.poll()) != null)

emitter.send(SseEmitter.event().name(e.name()).id(String.valueOf(e.id()))
                                       .data(e.payload(),
MediaType.APPLICATION_JSON));
        } catch (IOException | IllegalStateException ex) { close(); }
        finally { draining.set(false); if (!queue.isEmpty()) scheduleDrain();
}
    }
    void heartbeat() { try {
emitter.send(SseEmitter.event().comment("keepalive")); } catch (Exception ex)
{ close(); } }
    void close() { emitter.complete(); }
}
```

The emitter timeout is the token's remaining life plus 5 seconds, so the container never ends a stream before the hub has sent STREAM_CLOSING. For the public board the hub uses an expiry of 30 minutes. A client that stops reading blocks one sender thread until Tomcat's write timeout (connection-timeout, 5 seconds) fires; the 100-event queue and the 4 sender threads cap the harm, and the heartbeat finds dead clients.

3.13 Scheduled jobs

```
@Scheduled(fixedDelayString = "PT1M")
public void cancelUnpaid() {                                        // FR-ORD-
12
    Instant cutoff = clock.instant().minus(config.unpaidTimeout());
    int n;
    do { n = retry.run(() -> cancelService.cancelUnpaidBatch(cutoff, 50)); }
while (n == 50);
}

// OrderCancelService, one transaction per batch
@Transactional
public int cancelUnpaidBatch(Instant cutoff, int limit) {
    List<Long> ids = orders.claimUnpaid(cutoff, limit);              // S9:
FOR UPDATE SKIP LOCKED
    for (long id : ids) cancelBySystem(id, CancelReason.UNPAID_TIMEOUT);   //
same steps as a manual cancel
    return ids.size();
}
```

Job	Schedule	What it does
UnpaidOrderJob	Every minute	As above; restores stock and audits as SYSTEM
PurgeJobs	Hourly	Deletes idempotency_record rows older than 24 hours and refresh_token rows expired for 7 days, in batches of 1,000 rows (DELETE … WHERE id IN (SELECT id … LIMIT 1000))
LateTicketScanner	Every 10 seconds	Finds open tickets past their red threshold, publishes LATE_TICKET to the manager stream once per ticket (a Caffeine set remembers which, for one hour)

Every job catches and logs its own exceptions, so one failure never stops the schedule. All of them are safe with several instances because they claim rows with SKIP LOCKED or delete idempotently.

4. Module designs

Each module lists its classes and then the algorithms that carry risk. Routine CRUD follows the pattern of the first module that uses it: a controller method, a service method that validates and writes inside one transaction, an audit record, and the matching event.

4.1 auth

Class	Responsibility
AuthController	login, refresh, logout, me, change-password; checks the Origin header on the cookie endpoints
AuthService	login: the sequence below
LockoutService	Failure counter and lock on the user row (FR-AUTH-06)
RefreshTokenService	Issue, rotate, revoke family, revoke all for a user (Should)
UserAdminService	Create, patch, password reset, list; enforces SELF_LOCKOUT and LAST_ADMIN
AppUser	Entity with @Version; stations are held in user_station and read through a repository

```
public LoginResult login(String username, String password, String address) {
// not @Transactional
    loginLimiter.check(address, username);                       // LOGIN
bucket first: 429 even for unknown names
    AppUser user = users.findByUsernameIgnoreCase(username).orElse(null);
    if (user != null) lockout.requireNotLocked(user);            // 429 with
Retry-After while locked
    boolean matches = encoder.matches(password, user != null ?
user.getPasswordHash() : DUMMY_HASH);  // same cost either way
    if (user == null || !user.isActive() || !matches) {
        if (user != null) lockout.recordFailure(user.getId());   // its own
transaction: survives the 401
        throw ApiException.unauthenticated("Invalid username or password");
    }
    lockout.recordSuccess(user.getId());
    String token = jwt.issue(user, stationIdsOf(user));
    return new LoginResult(token, props.jwt().accessTtl(), UserView.of(user),
refreshTokens.issue(user.getId()));
}
```

LockoutService writes in a REQUIRES_NEW transaction. recordFailure locks the user row (SELECT FOR UPDATE), counts the failure (restarting at 1 when the last failure is older than 10 minutes), and sets locked_until to 15 minutes ahead when the count reaches 5. recordSuccess clears both. A DUMMY_HASH computed at start-up keeps the response time of an unknown username close to that of a known one.

RefreshTokenService stores only the SHA-256 of a 32-byte random token. Rotation looks the hash up and checks it, in this order: unknown or expired gives 401; consumed more than 10 seconds ago revokes the whole family and gives 401; consumed within the last 10 seconds gives 401 without revoking (two tabs); otherwise it marks the token consumed and issues a new one in the same family. Deactivating a user, resetting a password and changing a password call revokeAllForUser.

UserAdminService.update locks every active admin row (SELECT … FOR UPDATE) before it counts admins, so two admins deactivating each other at the same moment cannot leave the system with none (LAST_ADMIN).

4.2 menu

Class	Responsibility
MenuQuery	load(MenuKey): builds the POS MenuView from the database; loadForOrder(lines): the batched catalogue snapshot used to validate and price an order
MenuCache	Section 3.8; calls MenuQuery.load
MenuService	Admin changes to categories, items, stations, modifier groups, modifiers and availability; each change bumps menu_version and publishes MenuChanged
ComboService	Combos and their components (Should)

MenuQuery.loadForOrder always reads the database and never the cache (SAD principle P1). It collects the item, combo and modifier ids of the request and loads them in three queries: items with their category GST rate, activity and availability; their modifier groups and modifiers; combos with their components. The result is an immutable CatalogSnapshot.

```
@Transactional
public AvailabilityView setAvailability(long itemId, boolean available, Long
expectedVersion, Actor actor) {
    MenuItem item = items.lockById(itemId).orElseThrow(() ->
ApiException.notFound("item", itemId));
    if (!item.isActive()) throw
ApiException.validation(List.of(fieldError("itemId", "ITEM_INACTIVE")));
    if (available && item.isTrackStock() && stocks.quantityOf(itemId) == 0)
        throw ApiException.validation(List.of(fieldError("available",
"STOCK_ZERO")));
    if (expectedVersion != null && expectedVersion != item.getVersion())
throw ApiException.staleVersion(ItemView.of(item));
    if (item.isAvailable() == available) return AvailabilityView.of(item);
// no-op: no audit, version unchanged
    item.setAvailable(available);
// @Version increments on flush
    menuVersion.bump();
// UPDATE menu_version SET version = version + 1
    audit.record(actor.asActor(), AuditAction.ITEM_AVAILABILITY_CHANGED,
EntityType.MENU_ITEM, itemId,
                 String.valueOf(!available), String.valueOf(available), null,
Map.of());
    events.publishEvent(new MenuChanged());
    return AvailabilityView.of(item);
}
```

Deleting an item removes its item_station rows, modifier groups, modifiers and stock row, then the item, in one transaction. If a foreign key refuses (the item was ordered, is in a combo, or has stock history) the constraint handler returns 409 RESOURCE_IN_USE (section 3.3). That works only once kf_app holds DELETE on those tables (section 8).

4.3 inventory

Class	Responsibility
StockService	reserve, release, adjust, quantities
ItemStock, StockAdjustment	Entities; item_stock carries @Version
StockController	I-01, I-02, I-03

```
public void reserve(SortedMap<Long, Integer> qtyByItem) {            //
ascending item id, SAD 6.1
    Map<Long, Integer> reserved = new HashMap<>();
    for (var e : qtyByItem.entrySet()) {
        OptionalInt left = stockRepo.decrement(e.getKey(), e.getValue(),
clock.instant());   // S3 with RETURNING quantity
        if (left.isEmpty()) throw
ApiException.outOfStock(shortages(qtyByItem, reserved));   // the transaction
rolls back
        reserved.put(e.getKey(), e.getValue());
        if (left.getAsInt() == 0) events.publishEvent(new
StockDepleted(e.getKey()));        // handled inside this transaction
    }
}

private List<StockShortage> shortages(SortedMap<Long, Integer> wanted,
Map<Long, Integer> alreadyReserved) {
    Map<Long, Integer> now = stockRepo.quantities(wanted.keySet());
// one read
    // available = quantity now + what this transaction already took; a
shortage when below the request
}
```

The StockDepleted listener (an ordinary @EventListener, so it runs in the same transaction) locks the item row and sets available to false. The lock order is stock row first, then item row; setAvailability locks only the item row and reads the stock without locking, so the two cannot deadlock.

release adds quantities back (UPDATE … SET quantity = quantity + n), again in ascending item id order, for a cancel while CREATED, an unpaid timeout, or the negative side of an order edit. adjust applies a signed delta with a conditional update (the result must stay at or above zero, else NEGATIVE_STOCK), writes the StockAdjustment and audit record, and flips availability when the quantity crosses zero in either direction (FR-INV-03).

4.4 order

Class	Responsibility
OrderController	O-01 to O-08
OrderService	create

Class	Responsibility
OrderEditService	PUT /orders/{id}
OrderCancelService	cancel, cancelBySystem, cancelUnpaidBatch
OrderWorkflow	Row lock and compare-and-set transitions; implements OrderStatusPort for kitchen
OrderValidator	Shape and semantic validation, merged into one error list
PricingService	BR-03 arithmetic, pure and unit-tested
TokenService	S1
IdempotencyService	S2
OrderSearchService	O-02, with a keyset cursor
OrderStateMachine	Transition table with roles
Order, OrderLine	Entities; Order has @Version, lines are loaded with a fetch join

State machine. The sketch in SAD 16.3 is refined so that the two failures stay distinct: no rule for the pair of states gives 409, a rule that exists but excludes the role gives 403.

```
void check(OrderStatus from, OrderStatus to, Role actor) {
    List<Transition> forPair = RULES.stream().filter(t -> t.from() == from &&
t.to() == to).toList();
    if (forPair.isEmpty()) throw ApiException.invalidTransition(from.name(),
to.name(), null);   // 409
    if (forPair.stream().noneMatch(t -> t.roles().contains(actor))) throw
ApiException.forbidden(...);   // 403
}
```

So a Cashier cancelling an IN_KITCHEN order gets 403 (rule exists, MANAGER only), and anyone cancelling a READY order gets 409 (no rule), as API 9.5 requires.

Validation merges two sources. The API promises one 422 listing every violation (API 9.1), so POST and PUT /orders do not use @Valid on the controller. OrderValidator runs the Bean Validation constraints programmatically, then adds the checks that need the catalogue, and throws once.

```
public List<LineDraft> validate(OrderRequest req, CatalogSnapshot catalog) {
    List<FieldError> errors = new ArrayList<>(shape.validate(req));
// Bean Validation, run by hand
    List<LineDraft> drafts = new ArrayList<>();
    for (int i = 0; i < req.lines().size(); i++) {
        if (hasShapeErrors(errors, i)) continue;
// do not check a malformed line against the menu
        LineRequest l = req.lines().get(i);
        String p = "lines[" + i + "]";
        if ((l.itemId() == null) == (l.comboId() == null)) {
errors.add(err(p, "ONE_OF_REQUIRED")); continue; }
        if (l.itemId() != null) checkItemLine(l, p, catalog, errors, drafts);
        else checkComboLine(l, p, catalog, errors, drafts);
    }
    if (!errors.isEmpty()) throw ApiException.validation(errors);
// ITEM_UNAVAILABLE rule inside
    return drafts;
}
```

checkItemLine reports UNKNOWN_ID for a missing item, ITEM_UNAVAILABLE for an inactive or unavailable one, and for each modifier id UNKNOWN_ID, MODIFIER_NOT_ALLOWED (not in this item's groups), MODIFIER_UNAVAILABLE (inactive) or DUPLICATE_VALUE; it then counts the selections per group and reports MODIFIER_MIN or MODIFIER_MAX with the group name in the message. checkComboLine reports NOT_APPLICABLE for modifierIds and ITEM_UNAVAILABLE when any component is unavailable.

Pricing follows BR-03 and produces exactly the numbers the CHECK constraints demand:

```
public PricedOrder price(List<LineDraft> drafts) {
    List<PricedLine> lines = new ArrayList<>();
    Money subtotal = Money.ZERO, tax = Money.ZERO;
    int lineNo = 1;
    for (LineDraft d : drafts) {
        Money modifierTotal =
d.modifiers().stream().map(ModifierSnap::priceDelta).reduce(Money.ZERO,
Money::plus);
        Money lineSubtotal =
d.unitPrice().plus(modifierTotal).times(d.quantity());   // ck_ol_sub
        Money lineTax = lineSubtotal.percent(d.gstRate());
// ck_ol_tax
        lines.add(new PricedLine(lineNo++, d, modifierTotal, lineSubtotal,
lineTax, lineSubtotal.plus(lineTax)));
        subtotal = subtotal.plus(lineSubtotal);
        tax = tax.plus(lineTax);
    }
    return new PricedOrder(lines, subtotal, tax, subtotal.plus(tax));
// ck_orders_money
}
```

Idempotency implements API 2.7 on top of S2:

```
public Optional<Replay> begin(long userId, String key, String requestHash) {
    if (key == null) return Optional.empty();                          // no
key: nothing to replay
    int inserted = repo.insertIfAbsent(userId, key, requestHash);    // S2; a
duplicate waits for an in-flight first request
    if (inserted == 1) return Optional.empty();                        // first
use: carry on
    IdempotencyRow row = repo.find(userId, key).orElseThrow();
    if (!row.requestHash().equals(requestHash)) throw
ApiException.idempotencyKeyReused(row.createdAt());
    return Optional.of(new Replay(row.responseStatus(),
mapper.readValue(row.responseBody(), OrderView.class)));
}
public void complete(long userId, String key, int status, Object body, long
orderId) {
    if (key != null) repo.storeResponse(userId, key, status,
mapper.writeValueAsString(body), orderId);
}
```

The request hash is the SHA-256 of the request serialised with sorted properties, so field order and whitespace do not matter while the order of lines does.

Create is the transaction of SAD 7.1, written out:

```
@Transactional
public CreateResult create(OrderRequest req, String key, AuthenticatedUser
actor) {
    Instant now = clock.instant();
    Optional<Replay> replay = idempotency.begin(actor.id(), key,
hashOf(req));            // step 5
    if (replay.isPresent()) return new CreateResult(replay.get().view(),
true);

    CatalogSnapshot catalog = menuQuery.loadForOrder(req.lines());
// steps 6, 7: database, one batch
    List<LineDraft> drafts = validator.validate(req, catalog);
// throws with ALL errors
    PricedOrder priced = pricing.price(drafts);
// step 8
    stock.reserve(priced.trackedQuantities());
// step 9: ascending item id
    LocalDate day = businessDays.businessDayOf(now);
    int token = tokens.next(day);
// step 10: as late as possible
    Order order = orders.save(Order.create(priced, req, day, token,
orderNumber(day, token), actor.id(), now));
    audit.record(actor.asActor(), AuditAction.ORDER_CREATED,
EntityType.ORDER, order.getId(), null, "CREATED", null, Map.of());
    OrderView view = views.of(order);
    idempotency.complete(actor.id(), key, 201, view, order.getId());
// step 13: stored before commit
    return new CreateResult(view, false);
}

static String orderNumber(LocalDate day, int token) {
    return "KF-" + DateTimeFormatter.BASIC_ISO_DATE.format(day) + "-" +
String.format("%04d", token);   // five digits past 9,999
}
```

The controller wraps the call in the deadlock retry and turns the result into a response:

```
@PostMapping
ResponseEntity<OrderView> create(@RequestBody OrderRequest req,
        @RequestHeader(name = "Idempotency-Key", required = false) String
key,
        @AuthenticationPrincipal AuthenticatedUser user) {
    IdempotencyKeys.validate(key);                                        //
422 INVALID_FORMAT on the header
    CreateResult r = retry.run(() -> orderService.create(req, key, user));
    var response =
ResponseEntity.status(201).location(URI.create("/api/v1/orders/" +
r.view().id()));
    if (r.replayed()) response.header("Idempotency-Replayed", "true");
    return response.body(r.view());
}
```

Edit (OrderEditService) is a transaction that begins with the order lock: lock the order; refuse unless it is CREATED (409 INVALID_TRANSITION with operation EDIT); compare expectedVersion (409 STALE_VERSION); load the catalogue and validate as for create; price at current prices; compute the stock difference per tracked item (new quantity minus old); apply the differences in ascending item id order, reserving when positive and releasing when negative (a shortage is 409 OUT_OF_STOCK and the old reservation stays, because the transaction rolls back); replace the lines, modifiers and components; update totals; if nothing changed return the order untouched with no audit record; otherwise audit ORDER_EDITED with a summary. Token, order number and business day are never touched.

Cancel (OrderCancelService) shares one routine with the unpaid timeout:

```
@Transactional
public OrderView cancel(long orderId, CancelRequest req, AuthenticatedUser
actor) {
    Order order = workflow.lock(orderId);
// SELECT ... FOR UPDATE, SAD 6.3
    versions.checkIfGiven(req.expectedVersion(), order);
// STALE_VERSION
    OrderStatus before = order.getStatus();
    stateMachine.check(before, OrderStatus.CANCELLED, actor.role());
// 409 or 403
    order.cancel(req.reasonCode(), req.reasonText(), actor.id(),
clock.instant());   // status, reason, cancelled_by, cancelled_at
    List<TicketView> cancelled = ticketGateway.cancelOpenTickets(orderId,
clock.instant());  // NEW and IN_PROGRESS only; DONE stay DONE
    if (before == OrderStatus.CREATED)
stock.release(order.trackedQuantities());             // BR-12
    else refunds.createPendingIfPaid(orderId, actor);
// Should, FR-PAY-07
    audit.record(actor.asActor(), AuditAction.ORDER_CANCELLED,
EntityType.ORDER, orderId,
                 before.name(), "CANCELLED", req.reasonCode().name(),
Map.of());
    events.publishEvent(new OrderStatusChanged(summaries.of(order)));
    cancelled.forEach(t -> events.publishEvent(new TicketCancelled(t)));
    return views.of(order);
}
```

DONE tickets stay DONE when an order is cancelled: ck_ticket_done ties done_at to the DONE status, and food already made is wastage that the audit record shows.

OrderWorkflow is the only code that changes an order's status:

Method	Statement	Used by
lock(id)	SELECT ... FOR UPDATE through @Lock(PESSIMISTIC_WRITE)	Bump, recall, cancel, edit
markPaid(id, amount, now)	S5 with an added condition: AND total = :amount	PaymentService
markInKitchen(id, now)	Compare-and-set PAID to IN_KITCHEN	PaymentService

Method	Statement	Used by
markReady(order, actor)	S8, audit ORDER_READY, publishes OrderStatusChanged	TicketService.bump
returnToKitchen(order, actor)	READY to IN_KITCHEN	TicketService.recall

All native updates are declared with @Modifying(clearAutomatically = true, flushAutomatically = true), return the number of rows changed, increment version and set updated_at (DD-06). The lock timeout is the role setting of 2 seconds; the JPA lock-timeout hint does nothing on PostgreSQL and is not used.

Search (OrderSearchService) builds its SQL from the filters with named parameters. For cursor mode it uses a keyset condition on (created_at, id), newest first, and CursorCodec encodes the last row's values plus an HMAC so a tampered cursor is rejected with 400.

```
WHERE business_day = :day AND status IN (:statuses)
  AND (created_at, id) < (:cursorCreatedAt, :cursorId)       -- cursor mode
only
ORDER BY created_at DESC, id DESC
LIMIT :size
```

4.5 payment

Class	Responsibility
PaymentController	P-01 to P-06
PaymentService	pay: the payment-to-kitchen transaction
BillService	Builds the Bill, grouping GST by rate; provisional bill for a CREATED order; refuses a cancelled one
RefundService	createPendingIfPaid (called by cancel), complete (P-06)
Payment, Refund	Entities

```
@Transactional
public PaymentResult pay(long orderId, PayRequest req, AuthenticatedUser
actor) {
    Instant now = clock.instant();
    PaymentValidator.validate(req);                                      //
422: method, scale, range, reference, tendered
    if (workflow.markPaid(orderId, req.amount(), now) == 0)            // S5
with AND total = :amount; blocks if another payment is running
        throw explainFailedPayment(orderId);                             //
404, 409 or 422
    Payment payment = payments.save(Payment.of(orderId, req, actor.id(),
now));   // uq_payment_order is the backstop
    List<TicketRef> tickets = ticketGateway.createTickets(orderId, now);
// kitchen module, same transaction
    if (workflow.markInKitchen(orderId, now) == 0) throw new
IllegalStateException("order left PAID");  // cannot happen: we hold the row
    audit.record(actor.asActor(), AuditAction.PAYMENT_RECORDED,
EntityType.PAYMENT, payment.getId(), null, "RECORDED", null, Map.of());
    audit.record(actor.asActor(), AuditAction.ORDER_PAID, EntityType.ORDER,
orderId, "CREATED", "PAID", null, Map.of());
    audit.record(actor.asActor(), AuditAction.ORDER_IN_KITCHEN,
EntityType.ORDER, orderId, "PAID", "IN_KITCHEN", null, Map.of());
    publishTicketsAndStatus(orderId, tickets);
// listeners run after commit
    return PaymentResult.of(payment, orderRef(orderId, tickets));
}

private ApiException explainFailedPayment(long orderId) {
    Order o = orders.findById(orderId).orElseThrow(() ->
ApiException.notFound("order", orderId));   // 404
    return switch (o.getStatus()) {
        case CREATED   -> ApiException.amountMismatch(o.getTotal());
// 422: the total is not the amount sent
        case CANCELLED -> ApiException.invalidTransition("CANCELLED", "PAY",
null);   // 409
        default        -> ApiException.paymentAlreadyRecorded();
// PAID, IN_KITCHEN, READY, COMPLETED: 409
    };
}
```

Putting the amount into the compare-and-set closes a race: the state check and the amount check happen in one atomic statement, so an order edited between the client's read and the payment can never be paid at its old total. When two payments race, the second UPDATE waits for the first transaction, finds the status no longer CREATED, updates zero rows, and explainFailedPayment turns that into 409 PAYMENT_ALREADY_RECORDED.

A cash payment's change is computed by PaymentValidator (tendered minus amount) and stored, so ck_payment_cash can check it. BillService adds each line's GST to a map keyed by rate and verifies that the rows sum to taxTotal and subtotal before returning; a mismatch is an INTERNAL_ERROR, because the CHECK constraints should have made it impossible.

RefundService.createPendingIfPaid does nothing when the order has no payment; otherwise it inserts a REFUND_PENDING row, sets payment.status to REFUND_PENDING and audits REFUND_CREATED. complete locks the refund row, returns INVALID_TRANSITION when it is already REFUNDED, and otherwise sets REFUNDED on the refund and the payment.

4.6 kitchen

Class	Responsibility
TicketController	K-01 to K-06
KitchenStreamController	X-01 and X-02: authorise, then hub.subscribe
TicketService	start, bump, recall, acknowledge
TicketFactory	Routes order lines to stations and builds the tickets
KitchenTicketGateway	Implements order.spi.TicketGateway: createTickets, cancelOpenTickets
SlaCalculator	Age, percent of target and colour (BR-11)
TicketViewQuery	Loads tickets with items and modifier names in two queries (no N+1)
KitchenOverviewService	K-07 and the late-ticket query shared with LateTicketScanner

Class	Responsibility
Ticket	Entity with @Version and a domain method for each transition

Routing. One query returns a row for every item and station of the order. Combo lines have no item_id; their components supply it.

```
SELECT ol.id AS order_line_id,
       COALESCE(c.item_id,   ol.item_id)   AS item_id,
       COALESCE(c.item_name, ol.item_name) AS item_name,
       COALESCE(c.quantity, 1) * ol.quantity AS quantity,
       ist.station_id, mi.target_prep_seconds
FROM kf.order_line ol
LEFT JOIN kf.order_line_component c ON c.order_line_id = ol.id
JOIN kf.menu_item mi     ON mi.id = COALESCE(c.item_id, ol.item_id)
JOIN kf.item_station ist ON ist.item_id = mi.id
JOIN kf.station st       ON st.id = ist.station_id AND st.active
WHERE ol.order_id = :orderId
```

TicketFactory groups the rows by station. Each group becomes one Ticket whose target_seconds is the largest target_prep_seconds in the group (BR-11), with rush and business_day copied from the order, plus one TicketItem per row. Every order line must appear in at least one group; otherwise the factory throws IllegalStateException, the payment transaction rolls back, and nothing is stored (API 10.1). The tickets are inserted with saveAll, which batches because ticket and ticket_item use sequences.

cancelOpenTickets runs UPDATE kf.ticket SET status = 'CANCELLED', cancelled_at = :now, version = version + 1 WHERE order_id = :id AND status IN ('NEW', 'IN_PROGRESS') RETURNING id, then loads the views.

Ticket transitions live in the domain class, so no service can skip the rule:

```
void transitionTo(TicketStatus to, Instant now) {
    if (!status.canGoTo(to)) throw
ApiException.invalidTransition(status.name(), to.name(), null);
    switch (to) {
        case IN_PROGRESS -> { if (status == TicketStatus.NEW) startedAt =
now;          // start
                              else { doneAt = null; recallCount++; } }
// recall: ck_ticket_done needs done_at cleared
        case DONE        -> doneAt = now;
        case CANCELLED   -> cancelledAt = now;
        default          -> throw new IllegalArgumentException(to.name());
    }
    status = to;
}
// canGoTo: NEW to IN_PROGRESS; IN_PROGRESS to DONE; DONE to IN_PROGRESS; NEW
or IN_PROGRESS to CANCELLED
```

Bump is the SAD 16.4 sketch with the refinements of the API:

```
@Transactional
public BumpResult bump(long ticketId, long expectedVersion, AuthenticatedUser
actor) {
    long orderId = tickets.orderIdOf(ticketId).orElseThrow(() ->
ApiException.notFound("ticket", ticketId));  // plain read
    Order order = orderPort.lock(orderId);                          // SELECT
... FOR UPDATE: the order first (SAD 6.3)
    Ticket t = tickets.findById(ticketId).orElseThrow();            // read
after the lock
    stationAccess.require(actor, t.getStationId());                 // 403
for an unbound Kitchen user
    t.requireVersion(expectedVersion);                              // 409
STALE_VERSION with the current view
    t.transitionTo(TicketStatus.DONE, clock.instant());             // 409
INVALID_TRANSITION unless IN_PROGRESS
    entityManager.flush();                                          // so the
count below sees this change
    boolean becameReady = false;
    if (order.getStatus() == OrderStatus.IN_KITCHEN &&
tickets.countOpen(orderId) == 0) {   // S7
        orderPort.markReady(order, actor.asActor());                // S8,
audit ORDER_READY, OrderStatusChanged
        becameReady = true;
    }
    audit.record(actor.asActor(), AuditAction.TICKET_BUMPED,
EntityType.TICKET, ticketId, "IN_PROGRESS", "DONE", null, Map.of());
    events.publishEvent(new TicketChanged(views.of(t)));
    return new BumpResult(views.of(t), new OrderRef(orderId,
order.statusNow(), order.versionNow(), becameReady));
}
```

Recall takes the same first five steps, then checks the order and the window before it changes anything:

```
if (order.getStatus() == OrderStatus.COMPLETED) throw
ApiException.invalidTransition("COMPLETED", "RECALL", "ORDER_COMPLETED");
if (t.getStatus() == TicketStatus.DONE &&
now.isAfter(t.getDoneAt().plus(config.recallWindow())))
    throw ApiException.invalidTransition("DONE", "RECALL",
"RECALL_WINDOW_EXPIRED");        // BR-17
t.transitionTo(TicketStatus.IN_PROGRESS, now);                      // clears
done_at, adds 1 to recall_count
if (order.getStatus() == OrderStatus.READY) orderPort.returnToKitchen(order,
actor.asActor());   // clears ready_at too
```

Start is the same shape without the order checks, and acknowledge-cancel sets cancel_ack_at on a CANCELLED ticket (no change when already set; 409 when the ticket is not cancelled). Every action ends by publishing TicketChanged.

SLA. BR-11: green until 80 percent of the target, amber from 80 up to 100, red beyond 100.

```
public SlaView of(Ticket t, Instant at) {
    Instant end = t.getDoneAt() != null ? t.getDoneAt() : at;
    long age = Duration.between(t.getCreatedAt(), end).toSeconds();
    int percent = (int) (age * 100 / t.getTargetSeconds());
    SlaColour colour = percent > config.redPercent() ? SlaColour.RED
                     : percent >= config.amberPercent() ? SlaColour.AMBER :
SlaColour.GREEN;
    return new SlaView(age, percent, colour);
}
```

4.7 board

BoardService.snapshot() runs one query, SELECT token, status, ready_at FROM kf.orders WHERE status IN ('IN_KITCHEN','READY') ORDER BY business_day, token, which uses ix_orders_open. IN_KITCHEN tokens form the preparing list. READY tokens form the ready list ordered by ready_at, each with readySinceSeconds and highlight set when the wait exceeds board_ready_highlight_minutes. The snapshot is cached for 1 second. A listener on OrderStatusChanged publishes BOARD_CHANGED with a fresh snapshot to the board stream whenever the new status is IN_KITCHEN, READY, COMPLETED or CANCELLED. markReady sets ready_at and returnToKitchen clears it, so a recalled token goes back to preparing. Orders left open across the business-day cutoff are rare, and if two days' tokens ever coexist both are shown.

4.8 report

ReportService uses NamedParameterJdbcTemplate with the SQL of DBD section 8. Every method validates the range (from not after to, at most 92 days), runs behind the bulkhead, and executes in a read-only transaction that first raises the statement timeout.

```
public SalesReport sales(LocalDate from, LocalDate to, PaymentMethod method,
OrderType type) {
    ReportRange.validate(from, to);
    return bulkhead.run(() -> readOnly.execute(status -> {
// TransactionTemplate with readOnly = true
        jdbc.getJdbcOperations().execute("SET LOCAL statement_timeout =
'10s'");
        List<SalesRow> rows = jdbc.query(S10_SQL, params(from, to, method,
type), SalesRow::map);
        return SalesReport.of(from, to, clock.instant(), rows);
// totals are summed here from the rows
    }));
}

@Component
class ReportBulkhead {
    private final Semaphore permits = new Semaphore(2);
    <T> T run(Supplier<T> work) {
        boolean ok;
        try { ok = permits.tryAcquire(100, TimeUnit.MILLISECONDS); }
        catch (InterruptedException e) { Thread.currentThread().interrupt();
throw ApiException.rateLimited("REPORT_CONCURRENCY", 1); }
        if (!ok) throw ApiException.rateLimited("REPORT_CONCURRENCY", 1);
        try { return work.get(); } finally { permits.release(); }
    }
}
```

Report	Statement	Notes
R-01 sales	S10	Totals summed in Java
R-02 top items	S11	by=revenue changes only the ORDER BY
R-03 prep times	S12 per station; S12b per item	S12b is proposed in section 8
R-04 hourly	S13	Zero-filled to 24 rows in Java

Report	Statement	Notes
R-05 cancellations	S14 (proposed in section 8), LEFT JOIN so system cancellations count as role SYSTEM	

A statement that runs past 10 seconds raises SQLState 57014, which the handler turns into 503 SERVICE_BUSY with reason QUERY_TIMEOUT. CSV export (Could) is a second HttpMessageConverter selected by Accept: text/csv that writes the same row objects.

5. Sequence diagrams

The create-order flow is SAD section 7.1, step for step, and its code is in section 4.4. The two flows below are the ones where the order of events matters most.

5.1 Record a payment and push to the stations

Steps 2 to 8 run in one database transaction (FR-PAY-02). In the figure, Payment is PaymentService, Workflow is OrderWorkflow and Kitchen is KitchenTicketGateway. The listeners publish only after the commit (step 9), so a screen never sees a ticket whose transaction rolled back. If any step fails, nothing is stored and the order stays CREATED.

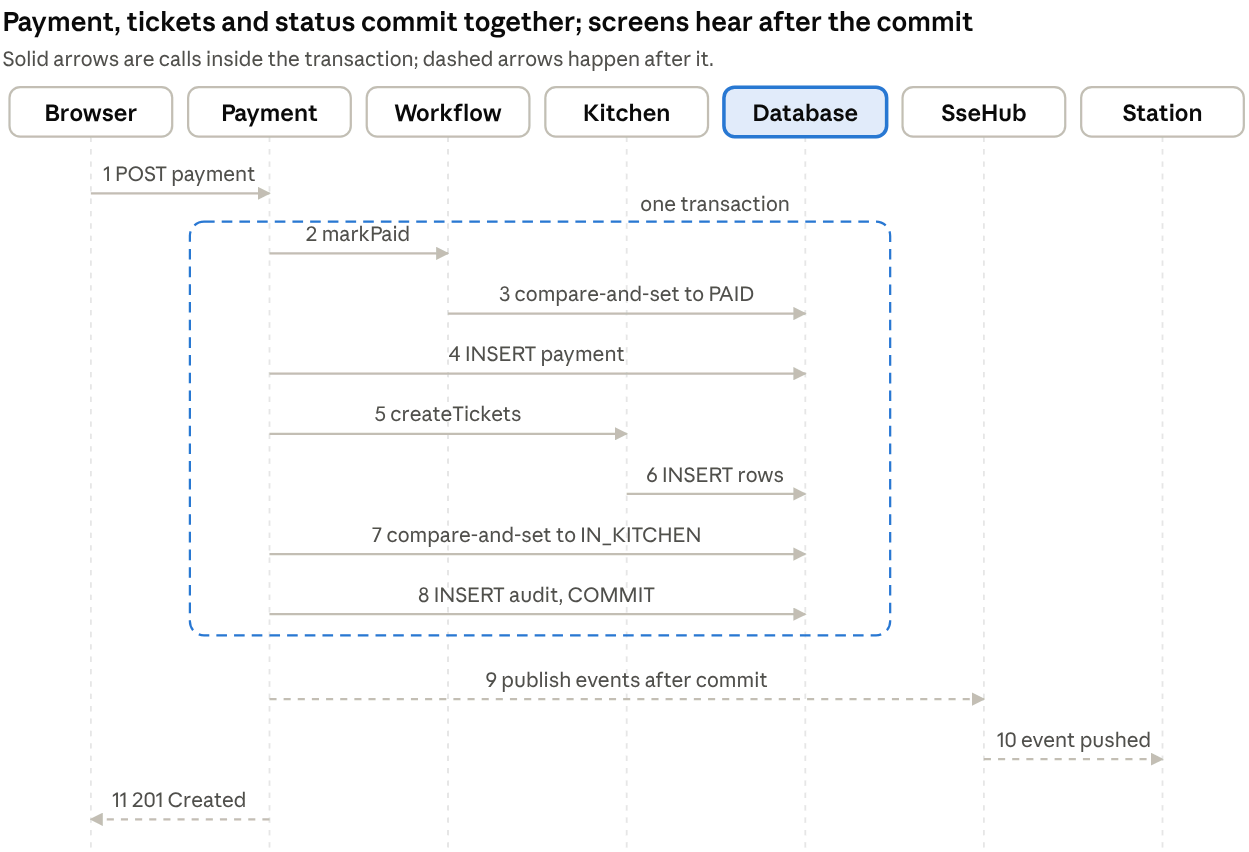


Figure 1. Recording a payment: the compare-and-set, the payment row, the tickets and the status change in one transaction, then the after-commit push.

5.2 Two cooks bump the last two tickets

The two transactions start together. Both want the order row first (SAD 6.3). Transaction A gets it, marks its ticket DONE, sees that one ticket is still open and commits. Transaction B then gets the lock, marks its own ticket DONE, sees none open and sets the order READY. Exactly one response carries becameReady true (NFR-CONC-04). The test for this is a mutation check: with the lock removed, the test must fail.

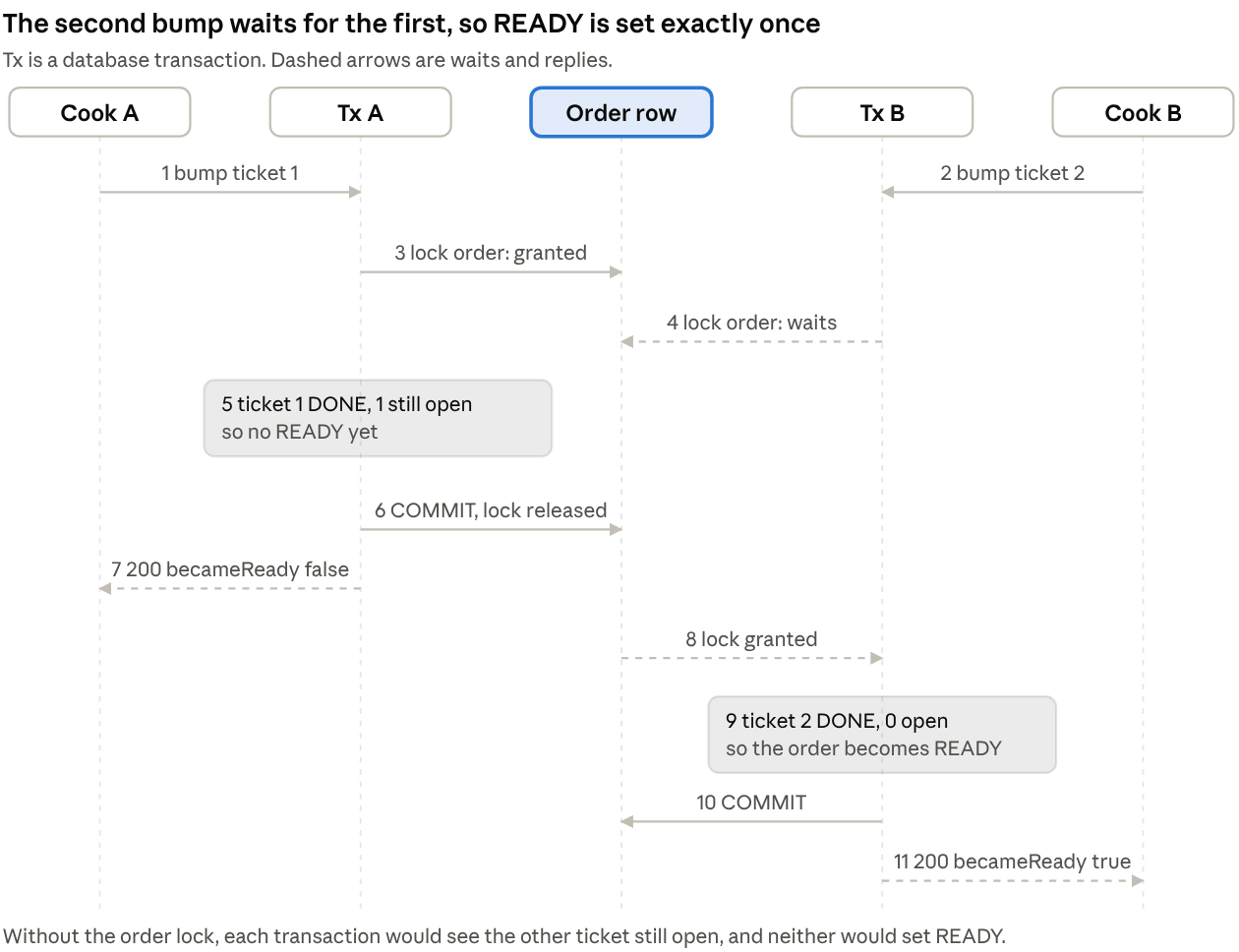


Figure 2. Two simultaneous bumps. The order lock makes the second transaction wait, so READY is set once and never missed.

6. Test design

Document 7 lists every test case; this section fixes how the tests are built so that the designs above can be proved.

6.1 Layers

Layer	What it covers	Tools	Notes
Unit	Money, PricingService, BusinessDayService, OrderStateMachine, ticket transitions, SlaCalculator, OrderValidator (with a fake catalogue), PasswordPolicy	JUnit 5, AssertJ	No Spring context; milliseconds each
Web slice	Strict JSON, the error envelope, the exception mapping tables, the URL role table	MockMvc	One test per row of the tables in sections 3.3 and 3.6
Integration	Services and repositories against real PostgreSQL with the real Flyway migration and the real roles	@SpringBootTest, Testcontainers	One shared container for the whole run
Concurrency	The invariants of SAD 6.2	Concurrently helper (6.2)	Repeated 50 times
Contract	API case tables	RestAssured or MockMvc against the running app	Section 16 of the API document
Architecture	Module and layer rules	ArchUnit	6.3
Load	NFR-PERF targets, no 429 at design workload	k6	CI smoke version, full run before the demo

6.2 Base classes and helpers

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@ActiveProfiles("test")
abstract class IntegrationTest {
    static final PostgreSQLContainer<?> PG = new PostgreSQLContainer<>
```

```
("postgres:16")
            .withInitScript("db/ops/roles-test.sql");        // creates
kf_migrator, kf_app and the schema
    static { PG.start(); }                                    // one
container for all test classes
    @DynamicPropertySource static void db(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", PG::getJdbcUrl);
        r.add("spring.flyway.user", () -> "kf_migrator");
        r.add("spring.datasource.username", () -> "kf_app");
    }
    @Autowired TestData data;                                 // builders:
user(role), item(price, stations), orderOf(...)
}

public final class Concurrently {                              // releases N
tasks together
    public static <T> List<Result<T>> run(int threads,
IntFunction<Callable<T>> task) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(threads);
        CountDownLatch ready = new CountDownLatch(threads), go = new
CountDownLatch(1);
        List<Future<Result<T>>> futures = new ArrayList<>();
        for (int i = 0; i < threads; i++) {
            Callable<T> c = task.apply(i);
            futures.add(pool.submit(() -> {
                ready.countDown(); go.await();
                try { return Result.ok(c.call()); } catch (Exception e) {
return Result.failed(e); }
            }));
        }
        ready.await(); go.countDown();
        List<Result<T>> results = new ArrayList<>();
        for (Future<Result<T>> f : futures) results.add(f.get(30,
TimeUnit.SECONDS));
        pool.shutdownNow();
        return results;
    }
}
```

A deterministic mutation test for the order lock. A probabilistic test can pass by luck, so the bump has one seam, a TestHooks bean whose afterTicketUpdate() does nothing in production. The test replaces it with a barrier that makes both transactions wait until

each has updated its own ticket, and replaces OrderWorkflow's lock with a no-op in the mutated variant:

- With the real lock, transaction B cannot reach the barrier while A holds the order row, so the barrier times out (500 ms, ignored) and the run proceeds in sequence: exactly one READY.
- With the lock removed, both reach the barrier, both update their tickets, both count one open ticket, and neither sets READY. The test asserts that the order is stuck in IN_KITCHEN, proving the lock is what prevents it.

The same pattern, a seam plus a variant with the protection removed, proves the stock conditional update (replace it with read-then-write and expect overselling), the payment compare-and-set and the idempotency insert.

6.3 Architecture rules

```
@AnalyzeClasses(packages = "com.kitchenflow", importOptions =
ImportOption.DoNotIncludeTests.class)
class ArchitectureTest {
    @ArchTest static final ArchRule apiDoesNotUseInfrastructure =

noClasses().that().resideInAPackage("..api..").should().dependOnClassesThat()
.resideInAPackage("..infrastructure..");
    @ArchTest static final ArchRule domainIsFreeOfWeb =

noClasses().that().resideInAPackage("..domain..").should().dependOnClassesTha
t()
            .resideInAnyPackage("org.springframework.web..",
"jakarta.servlet..");
    @ArchTest static final ArchRule featureModulesHaveNoCycles =
        slices().matching("com.kitchenflow.
(auth|menu|inventory|order|payment|kitchen|board|report|audit|config)..")
                .should().beFreeOfCycles();
    @ArchTest static final ArchRule orderDoesNotDependOnKitchen =

noClasses().that().resideInAPackage("com.kitchenflow.order..").should().depen
dOnClassesThat()
            .resideInAPackage("com.kitchenflow.kitchen..");
    @ArchTest static final ArchRule transactionsOnlyInApplicationLayer =

methods().that().areAnnotatedWith(Transactional.class).should().beDeclaredInC
lassesThat()
            .resideInAPackage("..application..");
}
```

6.4 Naming

API tests are named after the endpoint ID and the case row: O01_createsOrder_whenRequestIsValid, O01_returnsOutOfStock_whenLastUnitIsTaken, K04_setsReadyExactlyOnce_whenLastTwoTicketsAreBumped. A failing name then points straight at a row of the API document.

7. Build order

Work in this order. Each step starts with its tests and ends when the done condition holds; do not move on with a red build. Steps follow the roadmap phases of the product document, and the Must list comes before anything marked Should.

Step	Build	Read first	Tests first	Done when
Phase 0: skeleton				
1	Spring Boot project, Compose with PostgreSQL, roles.sql, Flyway V1 with all of DBD section 5	DBD 4 and 5; SAD 13	IntegrationTest base class	docker compose up starts the app; the schema checks of DBD 11.4 return zero rows
Phase 1: foundation				
2	Money, Clock, BusinessDayService, KitchenFlowProperties	3.1, 3.2, 2.3	Money rounding cases; the cutoff boundary	Unit tests green
3	Error model, GlobalExceptionHandler, JSON strictness, correlation filter, unknown-parameter interceptor	3.3 to 3.5; API 2 and 3	One web-slice test per mapping row	Every error comes back in the envelope
4	JwtService, SecurityConfig with the URL table, AuthenticatedUser	3.6; API 2.2	EndpointRuleCoverageTest; role matrix for the endpoints so far	A seeded user can log in and call an endpoint
5	auth module: login, lockout, me, user admin (U-01 to U-04)	4.1; API 6	Lockout on the sixth attempt; unknown user same response	Login rows of API 6.1 pass
6	seed_demo.sql and the user seed runner	DBD 10	Seed under 60 seconds; every item has a station (C6)	Demo data loads with one command
7	menu: GET /menu with cache and ETag, availability (M-17), then the admin catalogue	3.8, 4.2; API 7	ETag 304; cache race test	Menu cases of API 7.3 and 7.7 pass

Step	Build	Read first	Tests first	Done when
8	order create: OrderValidator, PricingService, TokenService, OrderService.create, GET orders	4.4; API 9.1 to 9.3	Pricing example 513.72; every validation code (AT-02)	O-01 case table passes, without idempotency and stock
9	OrderStateMachine, cancel, complete, edit, rush	4.4; API 9.4 to 9.6	Every legal and illegal transition by role	Order cases of API 9 pass
Phase 2: kitchen				
10	PaymentService with TicketFactory, bill	4.5, 4.6; API 10	AMOUNT_MISMATCH; AT-11 failure injection	AT-01 passes up to the kitchen
11	TicketService: start, bump, recall, acknowledge, K-01, K-02	4.6; SAD 6.3	The mutation test of 6.2; recall window; stale version	AT-05 and AT-06 pass
12	Domain events, SseHub, stream endpoints, board	3.11, 3.12, 4.7; API 12	Events only after commit; overflow closes; caps; token-expiry close	AT-09 passes; build the screens with polling first, then switch them to the stream
Phase 3: hardening				
13	IdempotencyService	4.4; API 2.7	Replay, key reuse, 50 concurrent duplicates	AT-03 passes
14	StockService, StockDepleted handling	4.3; API 8	100 concurrent orders for stock 10	AT-04 passes
15	AuditService everywhere, audit search	3.10; API 13.2	One audit record per transition (AT-01); audit write failure rolls back	FR-AUD cases pass
16	Rate limiting, LoginRateLimiter, config and board caches, DeadlockRetry	3.7 to 3.9; API 2.10	Every class: burst, refill, isolation	No 429 at design workload

Step	Build	Read first	Tests first	Done when
17	Concurrency suite for NFR-CONC-01 to 04	6.2	Each test repeated 50 times, each with its mutation variant	All pass; all variants fail as intended
18	Reports and the bulkhead	4.8; API 13	Ground-truth comparison on a known dataset	AT-10 passes
Phase 4: polish				
19	Nginx, Micrometer metrics, Dockerfile, CI gates, README, k6 smoke, 100,000-order generator and EXPLAIN checks	SAD 12, 13; DBD 7.2, 10.6	Plan assertions; coverage at 70 percent	One-command demo; CI green

Phase 5 (interview readiness) has no build steps: practise explaining sections 6.3 of the SAD, 5.2 of this document, and the three questions that mark this project out (duplicates, two cooks, the last item).

8. Traceability, proposed changes and next steps

8.1 Traceability

Design source	Implemented in
SAD 5.1 request pipeline	3.5, 3.6, 3.7
SAD 5.2 rate limiting	3.7, 4.1
SAD 5.3 caching	3.8, 4.7
SAD 5.4 load handling	2.3, 3.12, 4.8
SAD 6 concurrency and transactions	3.9, 4.3, 4.4, 4.5, 4.6, section 5
SAD 9 streams	3.11, 3.12, 4.6, 4.7
SAD 10 security	3.6, 4.1
SAD 11.3 jobs	3.13
DBD statements S1 to S14	4.3 to 4.8

Design source	Implemented in
API 2.4 error envelope	3.3
API 2.7 idempotency	4.4
API 2.8 versions	3.3, 4.6
API 2.10 limits	3.7

8.2 Proposed changes to the SAD, DBD and API

SAD (for v0.2)

1. Section 5.3: menuView holds one entry per filter combination (at most 64), not one entry.
2. Sections 5.3 and 4.7: the board snapshot is cached for 1 second.
3. Section 9: SNAPSHOT_REQUIRED carries reason SERVER_SHUTDOWN or OVERLOAD (when the event executor rejects work).
4. Section 16.3: replace the state machine sketch with the two-step check of 4.4, so a missing rule gives 409 and an excluded role gives 403.

DBD (for v0.2)

5. S5 gains the condition AND total = :amount, so the state check and the amount check are one atomic statement (4.5).
6. S3 returns the new quantity (RETURNING quantity), which lets the stock service detect depletion without a second read.
7. PUT /orders/{id} replaces lines, so kf_app needs DELETE on order_line, order_line_modifier and order_line_component. A guard trigger keeps that safe, and DD-10 is amended to allow it. The same guard is attached to the two child tables through their order_line.
8. Add statements S12b and S14.

```
-- S12b Kitchen cycle time per item (FR-RPT-03): each item takes the cycle
time of its ticket
SELECT ti.item_id, ti.item_name, count(*) AS tickets,
       avg(extract(epoch FROM t.done_at - t.created_at)) AS avg_seconds,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY extract(epoch FROM
t.done_at - t.created_at)) AS median_seconds,
       percentile_cont(0.9) WITHIN GROUP (ORDER BY extract(epoch FROM
```

```
t.done_at - t.created_at)) AS p90_seconds,
       avg((extract(epoch FROM t.done_at - t.created_at) >
t.target_seconds)::int) AS late_share
FROM kf.ticket t JOIN kf.ticket_item ti ON ti.ticket_id = t.id
WHERE t.business_day BETWEEN :from AND :to AND t.status = 'DONE'
GROUP BY ti.item_id, ti.item_name;

-- S14 Cancellations by reason and role (FR-RPT-05); system cancellations
count as SYSTEM
SELECT o.cancel_reason_code, COALESCE(u.role, 'SYSTEM') AS cancelled_by_role,
count(*) AS orders
FROM kf.orders o LEFT JOIN kf.app_user u ON u.id = o.cancelled_by
WHERE o.business_day BETWEEN :from AND :to AND o.status = 'CANCELLED'
GROUP BY o.cancel_reason_code, COALESCE(u.role, 'SYSTEM')
ORDER BY orders DESC;

-- Guard for change 7: lines can be deleted only while the order is CREATED
CREATE FUNCTION kf.guard_order_line_delete() RETURNS trigger LANGUAGE plpgsql
AS $$
BEGIN
  IF (SELECT status FROM kf.orders WHERE id = OLD.order_id) <> 'CREATED' THEN
    RAISE EXCEPTION 'order lines can be deleted only while the order is
CREATED';
  END IF;
  RETURN OLD;
END $$;
CREATE TRIGGER trg_order_line_delete_guard BEFORE DELETE ON kf.order_line
  FOR EACH ROW EXECUTE FUNCTION kf.guard_order_line_delete();
GRANT DELETE ON kf.order_line, kf.order_line_modifier,
kf.order_line_component TO kf_app;
```

API (for v0.2)

9. Section 2.5: authentication (401) is evaluated before the syntax checks (405, 415, 400), because Spring runs the filter chain first, and the URL-level 403 also comes before body parsing. Renumber the table accordingly.
10. Section 12.2: SNAPSHOT_REQUIRED reason OVERLOAD (change 3).

8.3 Open items

ID	Item	Decide in
OI-01	Confirm the library API forms (Bucket4j builder, Caffeine metrics binder, Spring Security JWT encoder) when versions are pinned	Phase 0
OI-02	Run CI smoke tests through a real Nginx container, or against the application alone	Phase 4
OI-03	Accept the order-line delete guard trigger, or soft-delete replaced lines instead	DBD v0.2
OI-04	JWT signing: HS256 with a shared secret is used (one service); RS256 would only matter with several verifiers	Closed unless services are split
OI-05	Idempotency replays return the stored JSON; recomputing from the order would be an alternative but would change a replay's body	API v0.2

8.4 Next step

Document 7, the Test Plan and Test Cases: strategy and environments, the complete case list derived from the SRS acceptance scenarios and every API case table, test data, tooling, entry and exit criteria, and the traceability matrix from requirement to test.

Version	Date	Author	Change
0.1	30 October 2026	Rathi	First draft